How we split the work

Goal. Build the full Wi-Fi TDoA localization chain in MATLAB as **5 independent blocks**. Each person owns one block, works alone, and plugs in at the end.

The idea

- Every block is a **black box**: fixed inputs in, fixed outputs out.
- You **assume** your inputs (make fake ones to test) and **promise** your outputs.
- Inputs come from the previous block; outputs go to the next block.
- Anything "known" (RX positions, bandwidth, ...) comes from one shared file: `config.m`.
- If the input/output contract is respected, integration is just calling the 5 functions in a row.

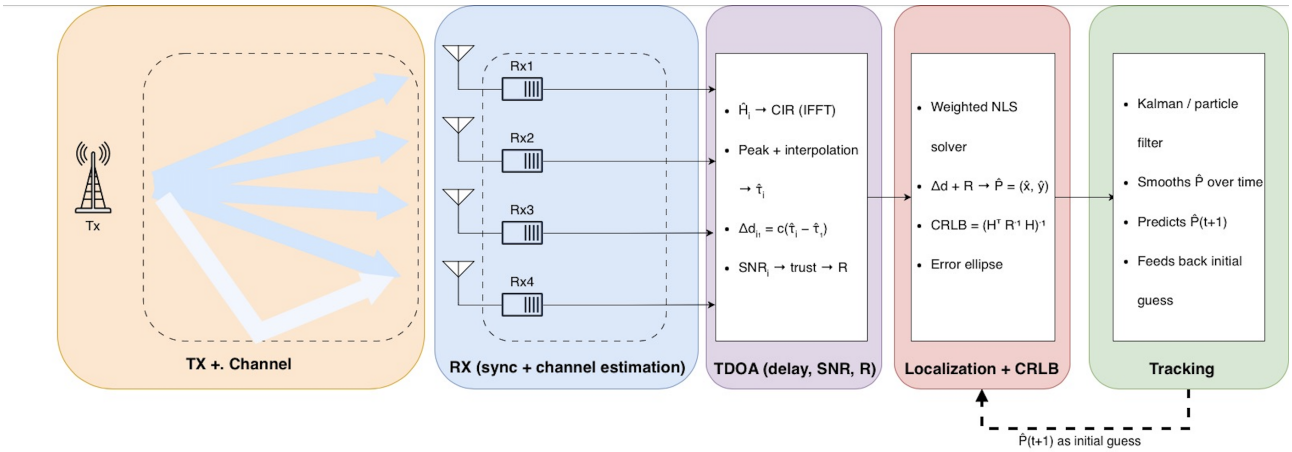
Rules everyone follows

- One main function per block, with the **exact name and signature** given on your block page.
- Read settings only from `cfg = config();`. Never hard-code numbers. Need a new setting? Ask the integrator to add it.
- SI units everywhere (m, s, Hz, W). Column vectors. RX order = rows of `cfg.Rx` (RX1 = reference).
- **Only Block 1** (and the final evaluation) may use the true TX position `cfg.P0` or the `truth` struct.
- Ship 3 things: your function, a test script with fake inputs, and one plot that shows it works.
- Work in your own folder on Git. Don't edit other blocks' files.

Shared config: `config.m`

Field	Meaning	Example	Used by
`c`	speed of light	`physconst('LightSpeed')`	1, 3
`fc`	carrier frequency	5.18e9 Hz (ch. 36)	1
`cbw`	Wi-Fi channel bandwidth	`'CBW20'`	1, 2, 3
`B`	bandwidth	20e6 Hz	1, 3
`fs`	sample rate	20e6 Hz	all
`Rx`	RX positions, 4×2	m	1, 4
`refIdx`	reference RX	1	3, 4
`P0`	true TX position, 2×1 (**Block 1 + evaluation only**)	m	1
`Pt_dBm`	TX power	0 dBm	1
`NF_dB`	RX noise figure	7 dB	1
`walls`	wall segments for bounces, one per row	`[x1 y1 x2 y2]`	1
`reflLoss_dB`	loss per wall bounce	6 dB	1
`zpFactor`	zero-padding factor for the CIR	16	3
`interp`	interpolation method	`'parabola'` / `'fft'`	3
`dt`	time between packets	0.1 s	5
`Nmc`	Monte Carlo trials	500	tests
`seed`	random seed	1	all

System overview



1. **TX + Channel:** sends a Wi-Fi packet to 4 receivers through simulated channels (delay, loss, bounce, noise).
2. **RX:** synchronizes once (on RX1), then estimates the channel at all 4 receivers.
3. **TDoA:** turns each channel estimate into an arrival time, then into time differences + trust matrix R.
4. **Localization + CRLB:** finds the TX position and its best-possible accuracy.
5. **Tracking:** smooths positions over time and feeds a prediction back to Localization.

Interface contract (what travels on each arrow)

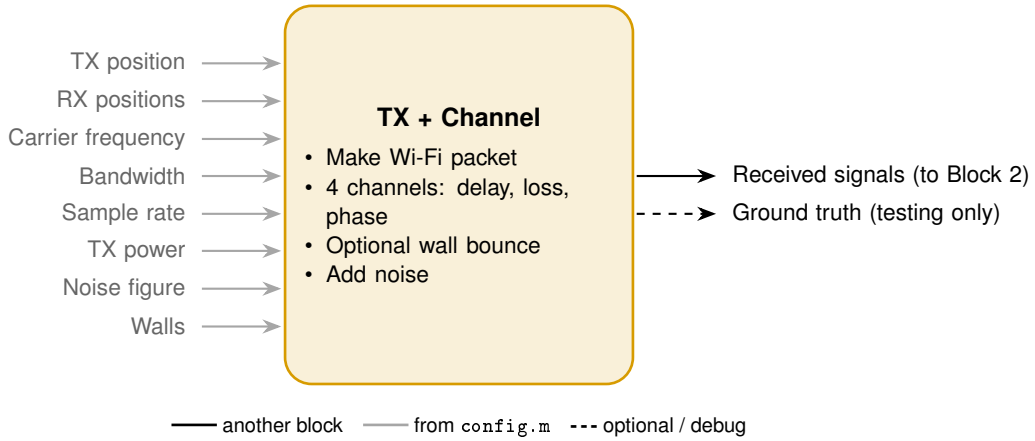
Arrow	Variable	Size	Unit	Meaning
1 → 2	`Y`	$L \times 4$	complex	received samples, one column per RX, same clock
1 → eval	`truth`	struct	–	true distances, delays, SNRs (**testing only**)
2 → 3	`Hhat`	52×4	complex	L-LTF channel estimate per active subcarrier, per RX
2 → 3	`noiseVar`	1×4	W	noise power estimate per RX
3 → 4	`deltad`	3×1	m	$d_i - d_1$ for RX2, RX3, RX4
3 → 4	`R`	3×3	m^2	covariance (trust) of `deltad`
4 → 5	`Phat`	2×1	m	estimated TX position $(\hat{x}, \hat{y})$
4 → 5	`Pcov`	2×2	m^2	position covariance (CRLB)
5 → 4	`Ppred`	2×1	m	predicted next position, used as initial guess

Function signatures (do not change)

```
[Y, truth]           = blk1_txChannel(cfg)
[Hhat, noiseVar]     = blk2_rx(Y, cfg)
[deltad, R, info3]   = blk3_tdoa(Hhat, noiseVar, cfg)
[Phat, Pcov]         = blk4_localize(deltad, R, cfg, Pinit)
[Ptrk, PtrkCov, Ppred, state] = blk5_track(Phat, Pcov, cfg, state)
```

Block 1 – TX + Channel

Owner: _____



Input / output format

Inputs (all from `config.m`):

- **TX position** P_0: one column, 2×1 $[x;\ y]$ in m (`cfg.P0`). Only this block uses it.
- **RX positions**: one matrix, 4×2, one row $[x\ y]$ per receiver, in m (`cfg.Rx`).
- **Carrier frequency, bandwidth, sample rate**: one number each, in Hz (`cfg.fc`, `cfg.B`, `cfg.fs`).
- **TX power** / **noise figure**: one number each, in dBm / dB (`cfg.Pt_dBm`, `cfg.NF_dB`).
- **Walls**: one matrix, $W \times 4$, one wall per row $[x_1\ y_1\ x_2\ y_2]$ in m; empty = no bounce (`cfg.walls`).

Outputs:

- **Received signals** $y_i(t)$: one complex matrix, $L \times 4$: L samples (rows) $\times$ 4 receivers (columns), same time axis (`Y`).
- **Ground truth**: one struct with true distances, delays and SNRs per RX. Testing only (`truth`).

Goal

Produce what 4 real receivers would capture from one Wi-Fi packet.

Implement

- Generate one full non-HT Wi-Fi packet (L-STF + L-LTF + L-SIG + short data). Block 2 needs L-STF to sync and L-LTF to estimate the channel.
- Add zeros before and after, plus one random start time shared by all RX (the unknown TX time).
- For each RX: delay by its distance (fractional, can fall between samples).
- For each RX: scale by path loss.
- Wall bounce (optional per RX): mirror the TX across the wall, add a second delayed, weaker copy.
- One common frequency offset for all RX (shared oscillator), random constant phase per RX.
- Add thermal noise, same level at every RX.
- Fill `truth`: distances, delays, received powers, SNRs, which RX has a bounce.

MATLAB tools

`wlanNonHTConfig`, `wlanWaveformGenerator`, `wlanSampleRate`, `delayseq` (or phase ramp in frequency), `fspl`, `comm.PhaseFrequencyOffset`, `awgn` / `wgn`, `physconst`.

Assume for now

- 2D, static TX, one packet per call.
- Free-space loss; walls are straight lines; at most one bounce per RX.

Test (standalone)

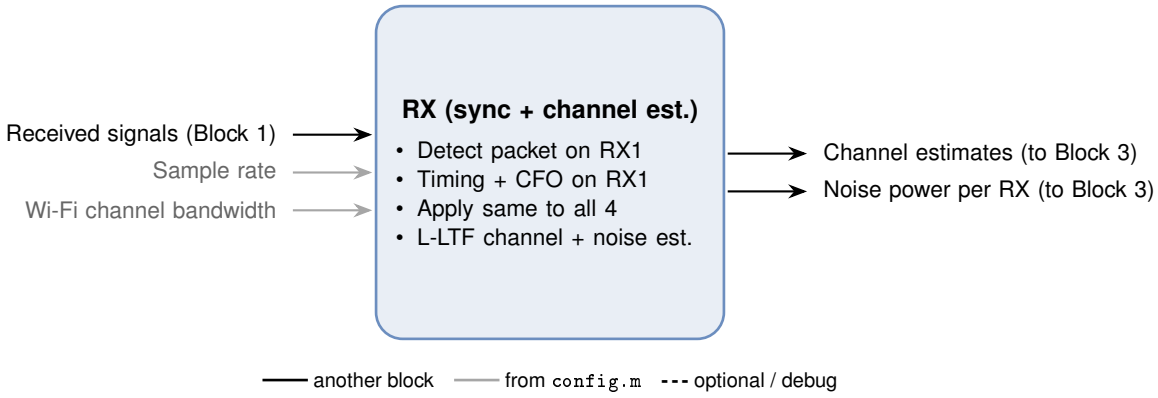
- Plot $|Y|$ for the 4 RX; mark true arrival times from `truth`.
- Turn noise off: shifts between RX must match distance differences.

Done when

`Y` has 4 columns at `cfg.fs`, all on the same time axis, and `truth` matches the geometry.

Block 2 – RX (sync + channel estimation)

Owner: _____



Input / output format

Inputs:

- **Received signals** $y_i(t)$: one complex matrix, $L \times 4$, samples $\times$ receivers (`Y`).
- **Sample rate**: one number in Hz (`cfg.fs`).
- **Wi-Fi channel bandwidth**: text, e.g. `'CBW20'` (`cfg.cbw`).

Outputs:

- **Channel estimates** $\hat{H}_i(k)$: one complex matrix, 52×4: 52 active subcarriers (rows) $\times$ 4 receivers (columns) (`Hhat`).
- **Noise power per RX** $\hat{\sigma}_i^2$: one row, 1×4, in W (`noiseVar`).

Goal

Turn raw samples into a clean channel estimate per receiver, **without destroying the time differences**.

Implement

- Packet detection on RX1 only.
- Coarse CFO (from L-STF), then fine CFO (from L-LTF), estimated on RX1.
- Symbol timing on RX1 $\rightarrow$ one start index.
- Apply that **same** start index and CFO correction to all 4 columns.
- Cut out the L-LTF, demodulate, estimate the channel per RX.
- Estimate noise power per RX.

Key rule

Never sync each RX separately. Per-RX timing removes exactly the delay differences Block 3 needs to measure.

MATLAB tools

`wlanPacketDetect`, `wlanFieldIndices`, `wlanCoarseCFOEstimate`, `wlanFineCFOEstimate`, `frequencyOffset`, `wlanSymbolTimingEstimate`, `wlanLLTFDemodulate`, `wlanLLTFChannelEstimate`, `wlanLLTFNoiseEstimate`.

Assume for now

- `Y` has one packet; all 4 columns share one clock.
- To test alone: build your own `Y` with `wlanWaveformGenerator` + simple known delays + noise.

Test (standalone)

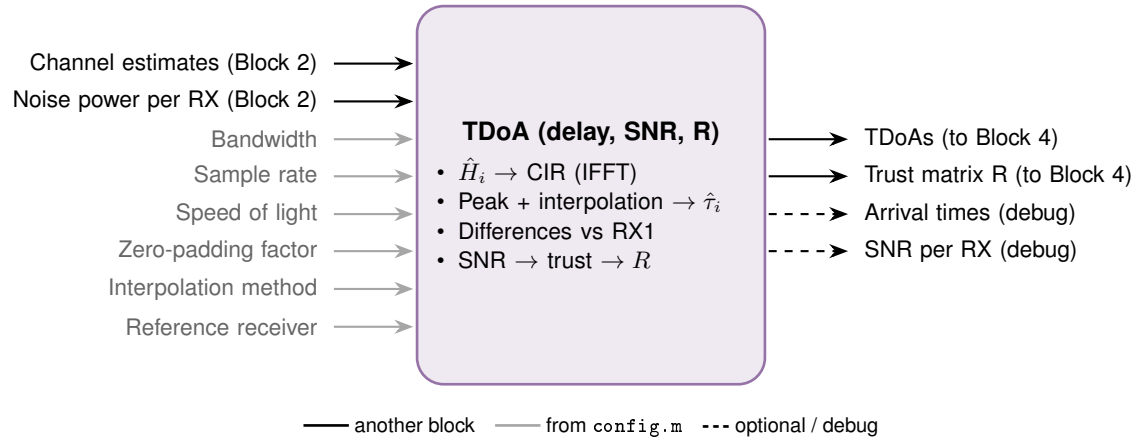
- Pure delay, no noise: phase of `Hhat` vs subcarrier is a straight line; steeper slope = later RX.
- Estimated noise matches the noise you added.

Done when

`Hhat` is 52×4, all columns referenced to one common timing, and `noiseVar` is close to the truth.

Block 3 – TDoA (delay, SNR, R)

Owner: _____



Input / output format

Inputs:

- **Channel estimates** $\hat{H}_i(k)$: complex matrix, 52×4, subcarriers $\times$ receivers (`Hhat`).
- **Noise power per RX** $\hat{\sigma}_i^2$: one row, 1×4, in W (`noiseVar`).
- **Bandwidth, sample rate, speed of light**: one number each, Hz / Hz / m/s (`cfg.B`, `cfg.fs`, `cfg.c`).
- **Zero-padding factor**: one integer, e.g. 16 (`cfg.zpFactor`).
- **Interpolation method**: text, `'parabola'` or `'fft'` (`cfg.interp`).
- **Reference receiver**: one integer, normally 1 (`cfg.refIdx`).

Outputs:

- **TDoAs** Δd_{i1}: one column, 3×1, in m: RX2, RX3, RX4 each minus RX1 (`deltad`).
- **Trust matrix** R: one symmetric matrix, 3×3, in m^2 (`R`).
- **Arrival times** $\hat{\tau}_i$ / **SNR per RX**: one column each, 4×1, in s / dB. Debug only (`info3.tauHat`, `info3.snrHat`).

Goal

Find when the signal arrived at each RX, take differences vs RX1, and say how much to trust each one.

Implement

- Place the 52 active subcarriers at their correct FFT positions (zeros at DC and guards).
- Zero-pad by `cfg.zpFactor`, IFFT $\rightarrow$ CIR per RX.
- Find the first strong peak (first path, not the strongest bounce).
- Refine between samples (`cfg.interp`).
- TDoA: difference vs RX1, convert to metres $\rightarrow$ `deltad`.
- SNR per RX from `Hhat` power and `noiseVar`.
- SNR + bandwidth $\rightarrow$ timing variance per RX $\rightarrow$ build R (include the shared RX1 term).

MATLAB tools

`wlanNonHTOFDMInfo` (active subcarrier indices), `ifft`, `ifftshift`, `interpft`, `max` / `findpeaks`.

Assume for now

- `Hhat` columns share one timing reference (Block 2's promise).
- To test alone: fake `Hhat` = pure delay per RX (linear phase) + noise.

Test (standalone)

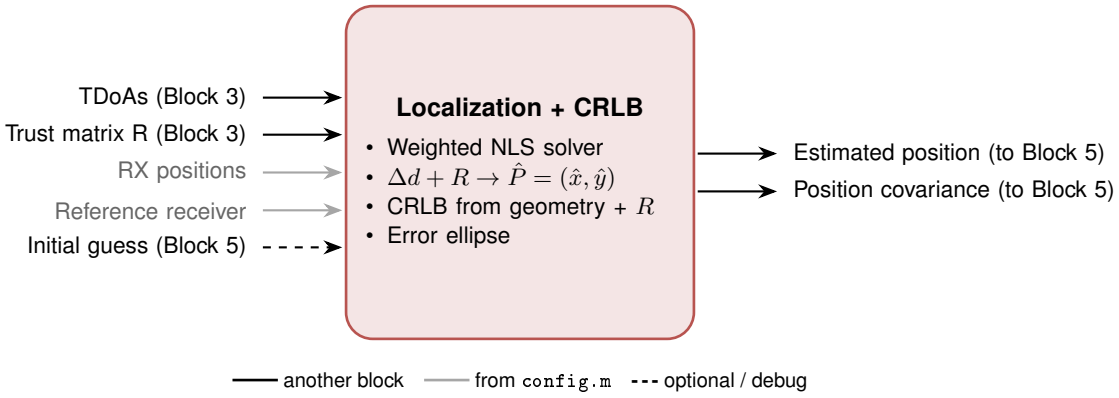
- No noise: delay error $\ll 1$ sample; with interpolation vs without.
- Monte Carlo: empirical covariance of `deltad` $\approx R$.
- Add a bounce: first-path detection still picks the direct path.

Done when

`deltad` matches truth within cm at good SNR, and R matches the measured spread.

Block 4 – Localization + CRLB

Owner: _____



Input / output format

Inputs:

- **TDoAs** Δd: one column, 3×1, in m (`deltad`).
- **Trust matrix** R: symmetric matrix, 3×3, in m^2 (`R`).
- **RX positions**: matrix, 4×2, one row per receiver, in m (`cfg.Rx`).
- **Reference receiver**: one integer, normally 1 (`cfg.refIdx`).
- **Initial guess**: one column, 2×1, in m; empty `[]` if none (`Pinit`).

Outputs:

- **Estimated position** $\hat{P}$: one column, 2×1 $[\hat{x};\ \hat{y}]$, in m (`Phat`).
- **Position covariance / CRLB**: symmetric matrix, 2×2, in m^2 (`Pcov`).

Goal

Find the TX position from the TDoAs, weighted by trust, and report the best accuracy possible.

Implement

- Reuse our existing `solveGradient.m` and `jacobianH.m`.
- Weight by R^{-1} (general R, not equal noise).
- Initial guess: `Pinit` if given, else centre of the RX.
- Update `crlb.m` for a general $R \to$ `Pcov`.
- Error ellipse from `Pcov` (plot helper).
- Report non-convergence instead of returning garbage.

MATLAB tools

Our solver files, `eig` (ellipse), optional cross-check with `lsqnonlin`.

Assume for now

- TDoA convention: `deltad(k)` = distance to RX$_{k+1}$ minus distance to RX1.
- To test alone: fake `deltad` from geometry + Gaussian noise with covariance R.

Test (standalone)

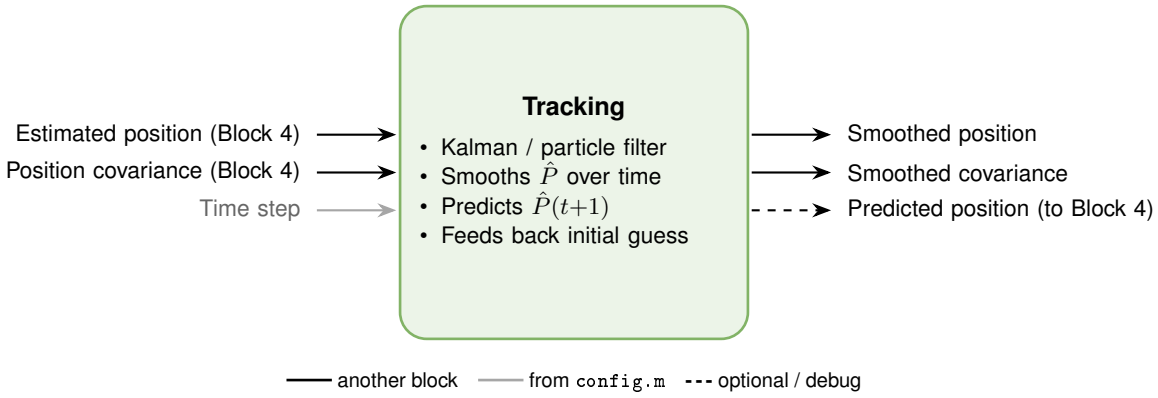
- No noise: exact position recovery.
- Monte Carlo: RMSE $\approx$ CRLB; plot points inside the ellipse.
- Try different RX layouts: see how geometry changes the ellipse.

Done when

RMSE tracks the CRLB across noise levels and the solver never silently diverges.

Block 5 – Tracking

Owner: _____



Input / output format

Inputs:

- **Estimated position** $\hat{P}(t)$: one column, 2×1, in m (`Phat`).
- **Position covariance**: symmetric matrix, 2×2, in m^2 (`Pcov`).
- **Time step**: one number, in s (`cfg.dt`).
- **Filter memory**: one struct passed back in on every call; start with `[]` (`state`).

Outputs:

- **Smoothed position**: one column, 2×1, in m (`Ptrk`).
- **Smoothed covariance**: symmetric matrix, 2×2, in m^2 (`PtrkCov`).
- **Predicted next position** $\hat{P}(t+1)$: one column, 2×1, in m; goes to Block 4 as its initial guess (`Ppred`).

Goal

Follow a moving TX over many packets and reduce error compared to single-shot positions.

Implement

- State: position + velocity; constant-velocity motion model.
- Measurement = `Phat`, measurement noise = `Pcov`.
- Each step: predict, then update.
- Output smoothed position + prediction for the next packet.
- Keep filter memory in `state` (passed in and out each call).
- Optional: particle filter version and compare.

MATLAB tools

Own Kalman filter, or `trackingKF` / `trackingPF` (Sensor Fusion and Tracking Toolbox).

Assume for now

- One position per packet every `cfg.dt` seconds.
- To test alone: make a TX path (line, circle), fake `Phat` = truth + noise drawn from `Pcov`.

Test (standalone)

- Plot truth vs raw `Phat` vs tracked `Ptrk`.
- Tracked RMSE $<$ raw RMSE.
- Tune process noise: too low lags turns, too high stops smoothing.

Done when

Tracked error is clearly lower than raw error on a moving path, and `Ppred` improves Block 4's starting point.

Integration

Folder structure

```
capstone-tdoa/
  config.m          shared settings (integrator owns it)
  main.m            runs the full chain
  blk1_txChannel/   blk1_txChannel.m   test_blk1.m
  blk2_rx/          blk2_rx.m          test_blk2.m
  blk3_tdoa/        blk3_tdoa.m        test_blk3.m
  blk4_localize/    blk4_localize.m    test_blk4.m
  blk5_track/       blk5_track.m       test_blk5.m
  results/
```

Full chain (`main.m`)

```
cfg = config();  rng(cfg.seed);
[Y, truth]       = blk1_txChannel(cfg);
[Hhat, noiseVar] = blk2_rx(Y, cfg);
[deltad, R]      = blk3_tdoa(Hhat, noiseVar, cfg);
[Phat, Pcov]     = blk4_localize(deltad, R, cfg, Ppred);
[Ptrk, PtrkCov, Ppred, state] = blk5_track(Phat, Pcov, cfg, state);
error = norm(Phat - cfg.P0);      % evaluation only
```

Integration order

1. Blocks 4 + 5 on fake data (already closest to done).
2. Plug Block 3 using fake `Hhat`.
3. Plug Blocks 1 + 2: first real end-to-end run.
4. Monte Carlo sweeps: bandwidth, SNR, RX layout, with/without wall bounce.

Before you hand over your block

- Signature and output sizes match the contract page exactly.
- Runs with `cfg = config();` and nothing else hard-coded.
- Test script runs alone and makes its plot.
- 3–5 line comment at the top: what it does, inputs, outputs.